Synthetic Training Data Augmentation Pipeline

Technical reference — synthetic training data for rail-sign recognition

Sodik — internal engineering documentation
Prepared 2026-08-20

Contents

1. Purpose and Scope

Real railway signage is heavily imbalanced in how often each sign type actually occurs in the field: some sign classes appear constantly along the network, while others show up only rarely, so real-world photographs of them are correspondingly scarce. A training dataset built purely from real photographs inherits this imbalance directly — common sign types end up with hundreds of examples, rare ones with a handful — and collecting enough additional real photographs of every rare sign type, under every lighting and weather condition the system will encounter in operation, is prohibitively slow and expensive on its own.

This pipeline exists to close that gap. It generates large volumes of photo-realistic synthetic training images from precise vector templates of each sign, composited onto genuine background footage and degraded to match real camera conditions, with per-class volume deliberately balanced (Section 8) so that rare sign types receive training coverage on par with common ones, rather than being underrepresented simply because of how infrequently they occur in the field.

The same mechanism also covers a case real photography cannot address at all: a newly introduced sign type, or a revised design for an existing one, for which no real-world photographs exist yet simply because it has only just entered service. Since every template starts as a hand-built SVG rather than a photograph, a new sign class can get a full, properly balanced set of training images (Section 8) from the moment its official design is available — well before enough real footage of it has had a chance to accumulate in the field.

2. The Template System

Each of the 16 sign classes is backed by one or more hand-built SVG vector templates encoding the sign's official geometry — proportions, stroke widths, colors, and (for text-bearing signs) digit placement. Several templates are built directly from dimensioned reference sheets for the relevant Czech railway signage standards, for example the ZT-4a "Rychlostník" speed-board standard and the ZT-53 "Kilometrická poloha" distance-marker standard, so template proportions are traceable to an authoritative source rather than estimated by eye.

Two categories of template exist:

- **Pictogram classes** (e.g. `way_prohibited`, `put_down_pantograf`) are represented by a single template, since the sign's appearance doesn't vary by printed value.
- **Numeric-value classes** (speed boards, distance markers, radio-channel markers) have many templates — a curated, representative sample of the printed values each sign type can plausibly show, not an exhaustive enumeration of every value in use on the real network — because the digit content is itself a major source of visual variation the augmented dataset needs to cover. Digits are rendered as real system-font glyphs (Helvetica Neue Bold) rather than hand-drawn shapes, so their proportions match ordinary printed typography.

A companion generation mode (`augment_number_templates.py`, Section 10) covers classes whose real-world value range is too large to enumerate as static files: rather than one template per possible number, each augmented variant renders its own fresh, independently random value at generation time.

2.1 Numeric-Value Coverage and Generalization

For these dynamically generated classes, each training variant's printed value is sampled randomly across the full plausible range for that sign type, rather than restricted to a hand-curated list of values already known to be in use on the real network. This is deliberate: sampling broadly across the range builds a dataset that captures the underlying visual pattern of how digits are rendered on that specific sign design — font, spacing, digit placement, plate proportions — rather than one that only ever shows a fixed set of specific values.

As a result, the augmented dataset already covers printed values that were never explicitly enumerated in the curated template list, including values that may appear for the first time in live footage after deployment. This coverage already works and is not a gap that needs to be closed for the numeric classes' augmentation to be considered complete.

Randomly sampled printed values across four numeric-value classes. Each row shares one visual pattern (font, digit placement, plate proportions) applied to a different randomly generated value.

> That said, if an authoritative, exhaustive list of every value that actually appears on the real rail network for a given sign type becomes available — for example the complete set of posted speed limits, distance markers, or radio-channel numbers actually in service — training coverage could be scoped to exactly that real-world set instead of a broader random range. This would remove any residual gap between the training distribution and the true deployed distribution, and is a straightforward, low-cost accuracy improvement if such a list exists.

Providing such a list is a low-cost change either way: the specific values each numeric-value class currently covers (Section 10.1) are a plain list in that class's generator script (for example, rychlostník's 37 speed values), not a hardcoded assumption baked into the pipeline. Adding, removing, or completely replacing the covered values — to match a supplied real-world list, or to extend coverage to new values as the network changes — only requires editing that list and re-running the generator; every downstream stage (augmentation, training) picks up the new templates automatically.

2.2 What Each Class Represents

Every one of the 16 classes corresponds to a specific, real railway sign. The table below gives a plain-language description of each, alongside how its templates are currently built.

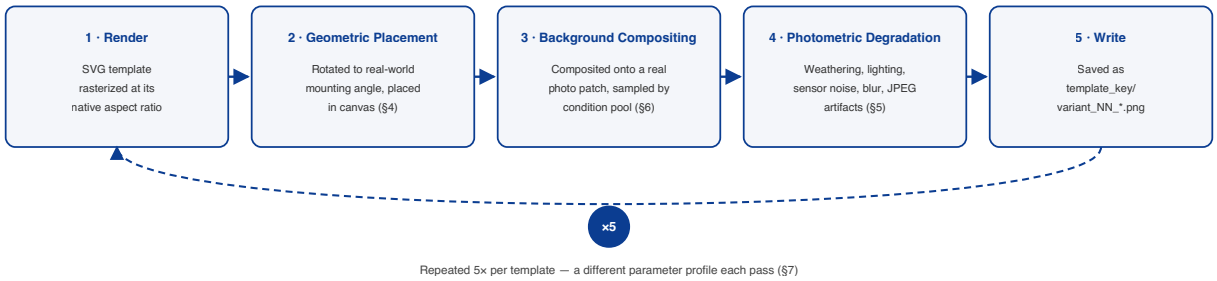
Class	What the sign indicates	Templates
begin_elevated_electricity	Start of a section with an elevated/hazardous overhead traction line	1 static pictogram
end_of_platform	End of a station platform	1 static pictogram
lift_pantograf	Instruction to raise the pantograph (the arm that collects power from the overhead line)	1 static pictogram
no_electricity	Section with no overhead electrical power; directional variants indicate the de-energized section lies ahead, left, or right	4 static pictograms (base + 3 directional)
prepare_for_semaphore	Advance warning of an upcoming semaphore/signal, in several line/triangle marking variants	11 static pictograms
prepare_low_electricity	Advance warning of a reduced-clearance electrified section ahead	1 static pictogram
prepare_turn_off_electricity	Advance warning to prepare to cut traction power	1 static pictogram
put_down_pantograf	Instruction to lower the pantograph	1 static pictogram
radiovník	Radio-channel marker: a handset pictogram plus the printed two-digit radio channel to use in that section	23 static, one per curated real channel value
rychlostník	Speed-restriction board: the permitted speed in km/h, single-line or stacked two-line (tilting-train speed over standard speed)	37 static, one per curated real speed value
sign_before_stop	Advance warning ahead of a stop signal or point	1 static pictogram
sklonovník	Gradient marker: the track's slope in per mille, increase/decrease direction, optionally with a secondary value	26 static, one per curated real gradient value
stanicník	Kilometer-position marker: the whole-kilometer distance plus a hectometer decimal, in split or comma layout	42 static, one per curated real position value
turn_off_electricity	Point at which traction power is actually cut	1 static pictogram
turn_on_electricity	Point at which traction power is restored	1 static pictogram
way_prohibited	Track/way closed to traffic	1 static pictogram

3. Pipeline Overview

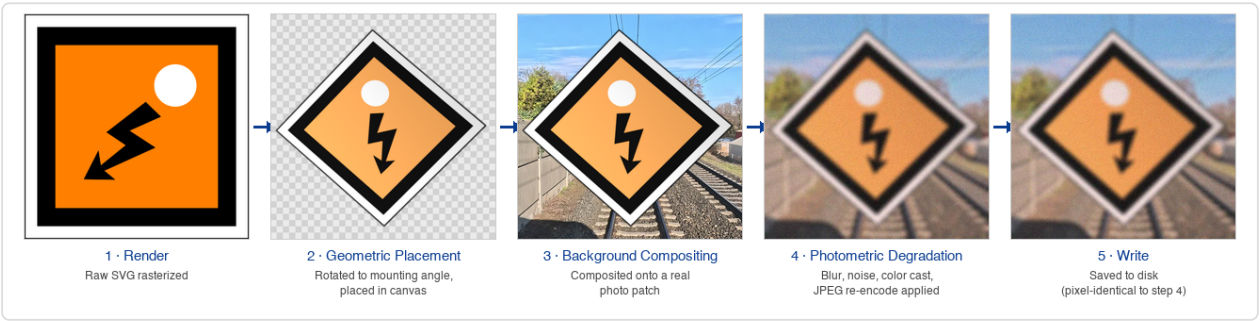
For every template, the pipeline executes the following stages, in order:

1. **Render.** The SVG template is rasterized at a resolution proportional to its aspect ratio.
2. **Geometric placement.** The rendered sign is rotated to its real-world mounting angle and composited into a canvas sized relative to its own bounding box (Section 4).
3. **Background compositing.** A randomly sampled patch of real camera footage, drawn from one of four weather/lighting condition pools, fills the canvas behind the sign (Section 6).
4. **Photometric degradation.** Material weathering, directional lighting, sensor noise, blur, and compression artifacts are applied to match real camera output (Section 5).
5. **Write.** The result is saved as `<template_key>/variant_<NN>_<condition>[_<pass>].png` under the shared output root.

This sequence runs five times per template with different parameter profiles — the five augmentation "passes" described in Section 7 — so every template contributes images spanning a deliberately wide range of real-world capture difficulty, not just a single idealized rendering.



The five pipeline stages executed for every template, run once per augmentation pass. Section references indicate where each stage is described in detail.



The same template, `begin_elevated_electricity`, carried through all five stages above. Step 2 is shown against a neutral checkered canvas (no real background yet) purely to make the rotation/placement step visible on its own; step 3 replaces that neutral canvas with an actual real background patch; step 5 is pixel-identical to step 4, since "write" only persists the result to disk.

4. Geometric Transformations

4.1 Mounting-angle rotation

Many signs are mounted at a fixed real-world angle relative to upright (a diamond-mounted pictogram, for instance, sits at 45°). Each template is rotated to its correct base mounting angle before any other processing, so the training data reflects how the sign is actually oriented in the field rather than however the vector artwork happens to be drawn.

4.2 Perspective and oblique-angle jitter

On top of the base mounting angle, a small random rotation and perspective warp (up to 14° rotation, up to 5% perspective distortion) is applied per variant, simulating normal camera-angle variation. A separate, lower-probability mode simulates a genuine raking viewing angle — one whole edge of the sign pushed toward its opposite edge, as when a flat sign is photographed from steeply off-axis — rather than only independent per-corner jitter, since these produce visually distinct distortion patterns and both occur in real footage.

4.3 Square-canvas convention

The canvas every sign is composited into is square, sized at a fixed multiple of the sign's own bounding box (1.05–1.15× for the square pass, matching the tight margin a detection pipeline typically produces when it crops a sign out of a full camera frame). Keeping the synthetic training crops on the same square-canvas convention as those real-world crops removes a class of train/inference mismatch that otherwise silently degrades accuracy on elongated sign shapes.

4.4 Aspect-ratio squashing

The low-visibility pass additionally squashes the sign's aspect ratio by a random factor (0.28–3.6×) before compositing, simulating the geometric distortion of a sign photographed at a steep viewing angle without a full perspective transform.

5. Photometric Transformations

5.1 Material weathering

With 35% probability, a variant's sign plate is desaturated and lightened in HSV space before compositing, simulating sun- and weather-bleached paint. Desaturation is applied in a color space where the sign's colored fills (orange, blue) fade far more visibly than its near-neutral white/black icon strokes and border — matching how real weathered signs stay legible even as their color washes out.

5.2 Directional lighting

With 45% probability, a linear brightness gradient (6–22% strength) is applied across the sign plate along a random direction, simulating angled sunlight raking across a reflective or embossed surface — brightening one edge, darkening the opposite. This is distinct from the uniform per-image brightness jitter applied later (Section 5.3), which shifts the whole frame's exposure rather than creating a gradient across the sign itself.

5.3 Sensor and compression degradation

Every variant, after background compositing, passes through a shared degradation function that applies, in order:

Effect	Detail
Brightness / contrast jitter	Brightness ±25, contrast randomized within a per-pass range
Color cast	Independent per-channel gain (0.92–1.08×), simulating white-balance variation
Hue rotation	Optional, widened for the low-visibility pass to cover off-color paint/lighting variants
Blur	Gaussian blur with sigma scaled to the image's own geometric-mean dimension (not a fixed pixel kernel), so elongated signs aren't over-blurred relative to their short axis. Parameter ranges were calibrated directly against measured sharpness (Laplacian variance) of real cropped sign photographs, since an earlier fixed-kernel version left synthetic images tens of times sharper than real camera crops of small, distant pictograms.
Sensor noise	Additive Gaussian noise, sigma randomized per pass
JPEG re-encoding	Randomized quality factor, reproducing real video-compression artifacts rather than only clean uncompressed renders

5.4 Low-resolution simulation

The low-resolution pass additionally downsamples the composited image to a small intermediate size (28–90px) and upsamples it back, reproducing the genuine information loss of a small or distant sign in the source video frame — not achievable by blur alone, since blur preserves high-frequency information that a true low-resolution capture never had.

6. Real-World Background Integration

Every augmentation pass draws its background from a pool of real camera footage, organized into four condition categories mirroring the operating conditions the live system encounters:

Condition	Description
`good_light`	Clear daytime footage
`dawn_fog_gloom`	Low-light dawn / overcast conditions
`evening_night`	Night driving footage
`rain`	Precipitation and wet-lens conditions

Where a background sequence carries its own genuine sign annotations, those specific frames are excluded from the sampling pool, so a real sign can never accidentally end up composited in as background clutter behind a synthetic one. A random patch, sized to the current canvas, is sampled from the relevant condition pool for every variant.

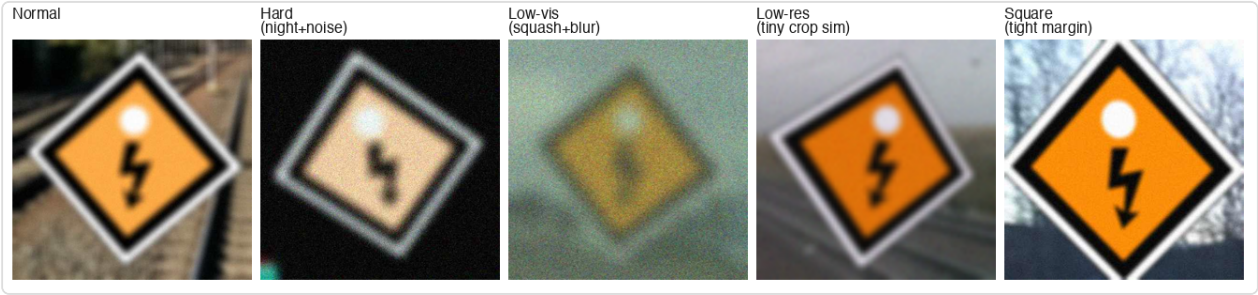
The same sign template composited onto real background footage from each of the four condition pools.

7. The Five Augmentation Passes

The transformations from Sections 4–6 are combined into five named passes per template, each with its own parameter profile:

Pass	Background pool	Distinguishing parameters	Default count
Normal	All 4 conditions, evenly split	Baseline geometric jitter and photometric degradation ranges	60 / template
Hard	Night only	Noise σ 15–40, 98% blur probability, JPEG quality 20–55	20 / template
Low-visibility	All 4 conditions	Aspect squash 0.28–3.6×, contrast 0.35–0.75, hue shift ±12°	20 / template
Low-resolution	All 4 conditions	Downscale/upscale round-trip through 28–90px	20 / template
Square	All 4 conditions	Canvas scale 1.05–1.15× the sign's own bounding box (tightest margin of the five passes, matching a typical real-world inference-time crop)	40 / template

At the default per-template multiplier, one template contributes **160 images** (60+20+20+20+40) across the five passes combined.



The same sign template rendered through all five passes.

8. Per-Class Volume Weighting

Sign classes differ in how many distinct real-world designs they cover: most pictogram classes have exactly one template, while a handful have several (for example, one base symbol plus directional-arrow variants). Left uncorrected, this produces an implicit imbalance in total training volume per class, since a class backed by several templates receives proportionally more training images than a single-template class — even though both are, from a downstream model's perspective, one output category competing on equal footing with every other.

The pipeline exposes a per-template multiplier table (`WEAK_TEMPLATE_MULTIPLIER`) that scales all five passes' variant counts for a given template, used to bring under-represented single-template classes up to volume parity with their multi-template counterparts. The current configuration applies a 3–4× multiplier to nine single-template classes, matching them to the volume of the best-represented comparable classes in the dataset (Section 9).

9. Current Dataset Composition

Class	Distinct templates	Training images
begin_elevated_electricity	1	480
turn_on_electricity	1	480
end_of_platform	1	640
lift_pantograf	1	640
no_electricity	4	640
prepare_low_electricity	1	640
prepare_turn_off_electricity	1	640
put_down_pantograf	1	640
sign_before_stop	1	640
turn_off_electricity	1	640
way_prohibited	1	640
prepare_for_semaphore	11	1,760
radiovník	23	3,680
sklonovník	26	4,160
rychlostník	37	5,920
stanicník	42	6,720
Total	**154**	**28,960**

At training time, this synthetic set is combined with a held-out-by-design real-photograph validation and test split: real crops are grouped by physical sign occurrence (consecutive video frames of the same sign share a track identifier) so no single physical sign's photographs can straddle the train/validation/test boundary.

The volumes above reflect the balance point tuned for the current sign set and its current per-class rarity profile. Nothing about the pipeline ties it to this specific distribution: the per-template multiplier mechanism described in Section 8 is a general-purpose configuration knob, not a one-off fix. A different sign set, a different downstream task, or a different target class distribution can be accommodated by re-tuning `WEAK_TEMPLATE_MULTIPLIER` (and, where relevant, adding new per-value templates) without redesigning the pipeline itself — the five-pass generation logic, background integration, and geometric/photometric transforms in Sections 4–7 stay identical regardless of how the per-class volumes are balanced.

10. Script Reference

The pipeline is implemented as a set of Python files, provided in full alongside this report under the project's `pipeline/` subfolder (all files sit side by side there, so every import between them is still a plain same-directory import -- no package path configuration is required to run any script directly). File names below are given without the `pipeline/` prefix for brevity.

10.1 Portability layer

Three small modules exist purely so this project has no dependency on anything outside itself:

File	Role
`config.py`	Every path used anywhere in the project, in one place. Each is overridable via an environment variable (`SIGN_TEMPLATES_DIR` , `SIGN_BACKGROUND_DIR` , `SIGN_AUGMENTED_OUTPUT_DIR`), defaulting to a folder inside the project itself.
`svg_render.py`	Loads SVG templates and rasterizes them to BGR pixel arrays via cairosvg -- a minimal, self-contained SVG manager with no dependency beyond cairosvg/ Pillow/OpenCV. Verified to produce pixel-identical output to the original SVG-rendering path this project's SVG manager was ported from.
`background_loader.py`	Lists and randomly samples real background photos from a folder per weather/ lighting condition, with an optional plain-text exclusion list (`annotated_frames.txt`) for any condition whose footage might contain a real sign somewhere in frame (see Section 6 and the `background_footage/README.md` that ships with this project).

10.2 Template generation

File	Produces
`generate_rychlostnik_templates.py`	Speed-restriction board templates (ZT-4a/ZT-60 standard), single-line and stacked two-line layouts, real printed digits
`generate_radiovnik_templates.py`	Radio-channel marker templates (handset pictogram + real printed two-digit number), built from a dimensioned reference sheet
`generate_stanicnik_templates.py`	Kilometer-position marker templates (ZT-53 standard), split and comma numeral layouts, with and without the yellow plate variant
`generate_sklonovnik_templates.py`	Gradient-marker templates, increase/decrease direction variants with optional secondary number

`digit_glyphs.py`	Shared procedural digit-glyph engine (stroke-chain based) used by the generators above where system-font text isn't the appropriate rendering method
`digit_glyphs_rounded.py`	Rounded-numeral glyph variant (true oval "0," curved-tail digits) -- currently unused by any generator (real system-font text ended up matching every current class's reference better), kept for a future sign category that might need hand-drawn rounded digits

Each script writes its output as one `.svg` file per template directly under `<config.TEMPLATES_DIR>/<class_name>/`. Every script's curated list of specific values (e.g. which speed values get a template) is a plain, easily edited Python list near the bottom of that file -- see the README for how to change it.

10.3 Augmentation pipeline

`augment_templates.py` is the main orchestrator described throughout this document. Key entry points:

Function	Role
`load_background_pools()`	Loads and caches real background footage for all 4 condition pools
`random_background_patch()`	Samples a random background patch of the requested canvas size
`warp_sign_onto_background()`	Applies mounting-angle rotation and perspective/oblique jitter, composites onto the background (Section 4)
`fade_sign()` / `add_directional_shading()`	Material weathering and directional lighting (Section 5.1–5.2)
`photo_degrade()`	Brightness/contrast/color-cast/blur/noise/JPEG degradation (Section 5.3)
`squash_aspect_ratio()` / `simulate_low_resolution()`	Low-visibility and low-resolution pass-specific transforms (Section 5.4)
`generate_variants()`	Runs the full per-variant pipeline for a given count/parameter profile; called once per pass per template
`main()`	Iterates every template under `config.TEMPLATES_DIR`, applies `WEAK_TEMPLATE_MULTIPLIER`, and runs all five passes for each

Key module-level constants: `CANVAS_SCALE` (base canvas sizing), `VARIANTS_PER_TEMPLATE` / `HARD_VARIANTS_PER_TEMPLATE` / `LOWVIS_VARIANTS_PER_TEMPLATE` / `LOWRES_VARIANTS_PER_TEMPLATE` / `SQUARE_VARIANTS_PER_TEMPLATE` (default per-pass counts), and `SQUARE_CANVAS_SCALE_RANGE` (the tight-margin range used by the square pass).

`WEAK_TEMPLATE_MULTIPLIER` ships empty in this project -- fill it in based on your own class list and template counts (Section 8).

`augment_number_templates.py` is a companion script for classes whose real-world value range is too large to enumerate as static files (Section 2). It imports and reuses every transform function from `augment_templates.py` unchanged, but generates a fresh, independently random template at variant-generation time rather than reading a fixed file. Not run by default -- see Section 2.1.

Output convention (both scripts):

`<config.OUTPUT_DIR>/<template_key>/variant_<NN>_<condition>[_<pass_suffix>].png`
(defaults to `./output/augmented_templates/...` inside the project)

Usage:

`pip install -r requirements.txt` (one-time setup)
`python pipeline/generate_<class>_templates.py` (template generation, per class, as needed)
`python pipeline/augment_templates.py` (regenerates every template's augmented images)
`python pipeline/augment_number_templates.py` (optional, dynamic-value classes)